

Assignment-13.4

T.Ramyasri
2303A52136

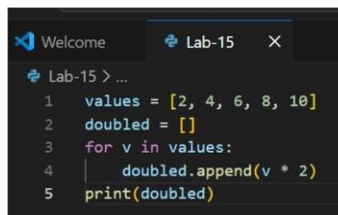
Batch 41

Task-1: Refactoring Data Transformation Logic

PROMPT:

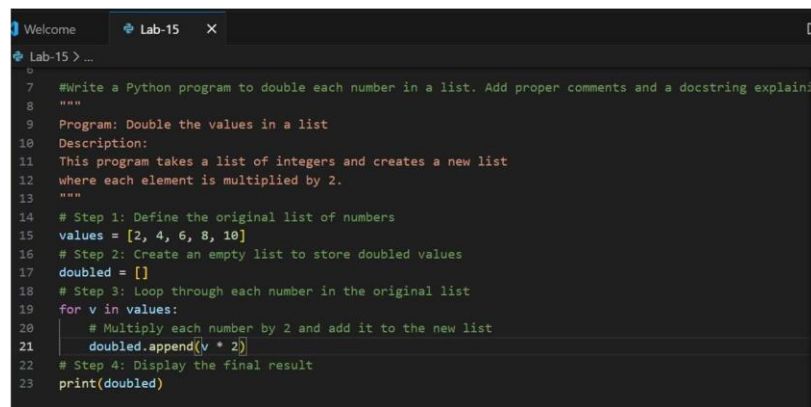
Write a Python program to double each number in a list. Add proper comments and a docstring explaining the program.

Legacy Code:



```
1 values = [2, 4, 6, 8, 10]
2 doubled = []
3 for v in values:
4     doubled.append(v * 2)
5 print(doubled)
```

Refactored Code:



```
7 #Write a Python program to double each number in a list. Add proper comments and a docstring explaining the program.
8 """
9 Program: Double the values in a list
10 Description:
11 This program takes a list of integers and creates a new list
12 where each element is multiplied by 2.
13 """
14 # Step 1: Define the original list of numbers
15 values = [2, 4, 6, 8, 10]
16 # Step 2: Create an empty list to store doubled values
17 doubled = []
18 # Step 3: Loop through each number in the original list
19 for v in values:
20     # Multiply each number by 2 and add it to the new list
21     doubled.append(v * 2)
22 # Step 4: Display the final result
23 print(doubled)
```

OUTPUT:

```
[4, 8, 12, 16, 20]
```

EXPLANATION:

This program creates a list of numbers [4,8,12,16,20] and then uses a loop to go through each number one by one. For every number in the list, it multiplies the number by 2 and adds the result to a new list called doubled. After the loop finishes, the program prints the new list, which contains all the original numbers multiplied by 2.

Task-2: Improving Text Processing Code Readability

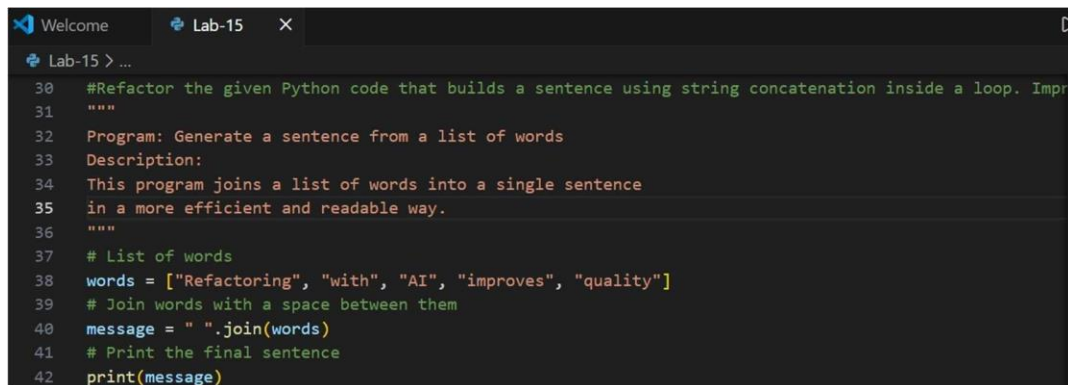
PROMPT:

Refactor the given Python code that builds a sentence using string concatenation inside a loop. Improve efficiency and readability while keeping the output unchanged.

Legacy Code:

```
22
23 words = ["Refactoring", "with", "AI", "improves",
24 "quality"]
25 message = ""
26 for w in words:
27     message += w + " "
28 print(message.strip())
```

Refactored Code:



```
30 #Refactor the given Python code that builds a sentence using string concatenation inside a loop. Impr
31 """
32 Program: Generate a sentence from a list of words
33 Description:
34 This program joins a list of words into a single sentence
35 in a more efficient and readable way.
36 """
37 # List of words
38 words = ["Refactoring", "with", "AI", "improves", "quality"]
39 # Join words with a space between them
40 message = " ".join(words)
41 # Print the final sentence
42 print(message)
```

OUTPUT:

```
Refactoring with AI improves quality
```

EXPLANATION:

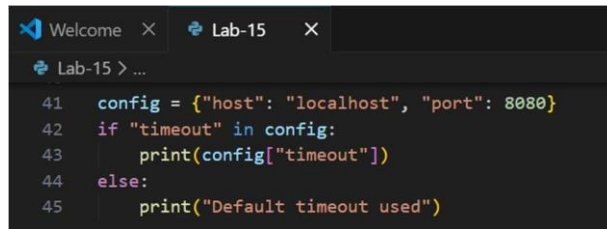
The original code used a loop to add each word to a string one by one, which is less efficient because strings are immutable in Python. Every time a word is added, a new string is created. The improved version uses the `join()` method, which combines all the words in the list into a single string with spaces in between. This method is faster, cleaner, and easier to read while producing the same output: Refactoring with AI improves quality.

Task-3: Safer Access to Configuration Data

PROMPT:

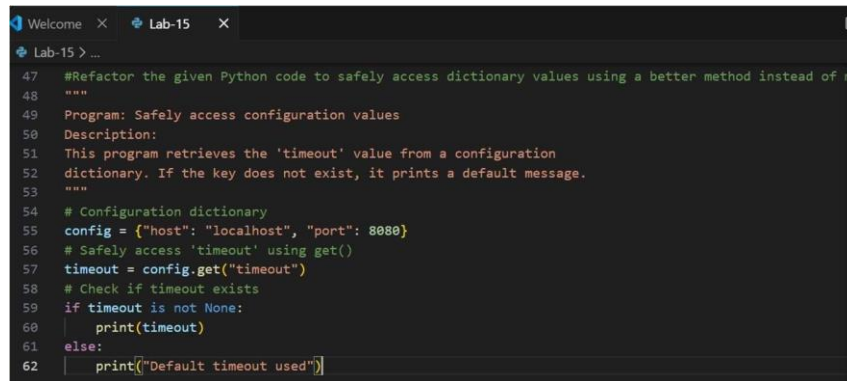
Refactor the given Python code to safely access dictionary values using a better method instead of manually checking for keys. Keep the output unchanged if the key is missing.

Legacy Code:



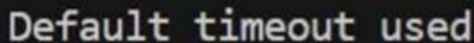
```
41 config = {"host": "localhost", "port": 8080}
42 if "timeout" in config:
43     print(config["timeout"])
44 else:
45     print("Default timeout used")
```

Refactored Code:



```
47 #Refactor the given Python code to safely access dictionary values using a better method instead of m
48 """
49 Program: Safely access configuration values
50 Description:
51 This program retrieves the 'timeout' value from a configuration
52 dictionary. If the key does not exist, it prints a default message.
53 """
54 # Configuration dictionary
55 config = {"host": "localhost", "port": 8080}
56 # Safely access 'timeout' using get()
57 timeout = config.get("timeout")
58 # Check if timeout exists
59 if timeout is not None:
60     print(timeout)
61 else:
62     print("Default timeout used")
```

OUTPUT:



```
Default timeout used
```

EXPLANATION:

The original code manually checks whether the key "timeout" exists in the dictionary before accessing it. This works but makes the code longer. The improved version uses the `get()` method, which safely retrieves the value of a key. If the key does not exist, it returns a default value instead of causing an error. This makes the code shorter, cleaner, and safer while keeping the same output: Default timeout used.

Task-4: Refactoring Conditional Logic for Scalability

PROMPT:

Refactor the given Python code that uses multiple if-elif conditions to perform operations. Replace it with a cleaner and more scalable mapping-based approach while keeping the same output.

Legacy Code:

```
61 action = "divide"
62 x, y = 10, 2
63 if action == "add":
64     result = x + y
65 elif action == "subtract":
66     result = x - y
67 elif action == "multiply":
68     result = x * y
69 elif action == "divide":
70     result = x / y
71 else:
72     result = None
73 print(result)
```

Refactored Code:

```
75 #Refactor the given Python code that uses multiple if-elif conditions to perform operations. Replace
76 """
77 Program: Perform mathematical operations using mapping
78 Description:
79 This program performs a mathematical operation based on
80 the given action using a dictionary mapping approach.
81 """
82 # Given inputs
83 action = "divide"
84 x, y = 10, 2
85 # Define operations using lambda functions
86 operations = {
87     "add": lambda a, b: a + b,
88     "subtract": lambda a, b: a - b,
89     "multiply": lambda a, b: a * b,
90     "divide": lambda a, b: a / b
91 }
92 # Get the result safely
93 operation = operations.get(action)
94 result = operation(x, y) if operation else None
95 # Print result
96 print(result)
```

OUTPUT:

5.0

EXPLANATION:

The original code used multiple if-elif conditions to check which operation to perform. While it works, it becomes harder to maintain when more operations are added. The

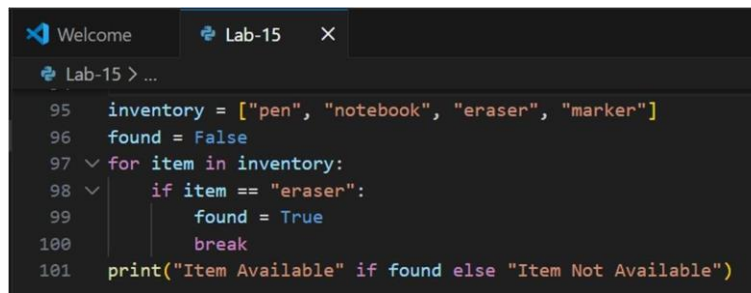
refactored version uses a dictionary called operations that maps action names to small functions (lambda functions). This makes the code cleaner and easier to expand. If a new operation is needed, we can simply add it to the dictionary instead of adding more if-elif statements. The functionality remains the same, and the output is still 5.0.

Task-5: Simplifying Search Logic in Collections

PROMPT:

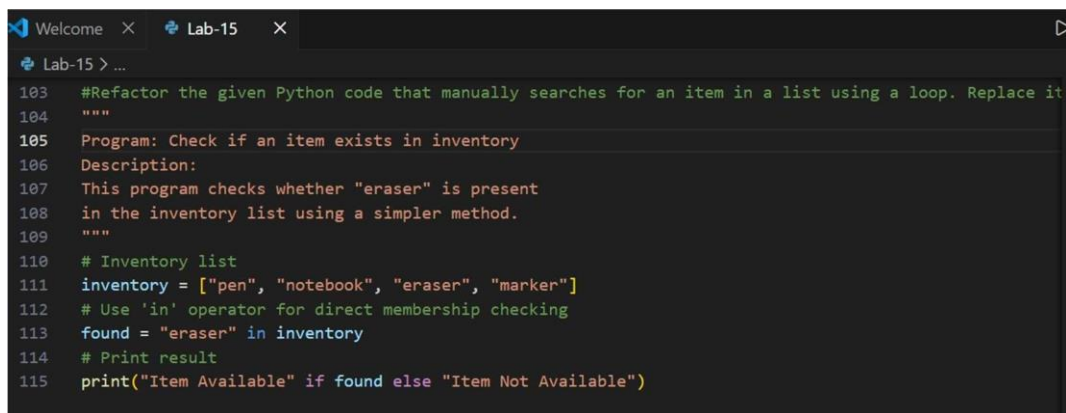
Refactor the given Python code that manually searches for an item in a list using a loop. Replace it with a more concise and readable approach while keeping the same output.

Legacy Code:



```
95 inventory = ["pen", "notebook", "eraser", "marker"]
96 found = False
97 for item in inventory:
98     if item == "eraser":
99         found = True
100         break
101 print("Item Available" if found else "Item Not Available")
```

Refactored Code:



```
103 #Refactor the given Python code that manually searches for an item in a list using a loop. Replace it
104 """
105 Program: Check if an item exists in inventory
106 Description:
107 This program checks whether "eraser" is present
108 in the inventory list using a simpler method.
109 """
110 # Inventory list
111 inventory = ["pen", "notebook", "eraser", "marker"]
112 # Use 'in' operator for direct membership checking
113 found = "eraser" in inventory
114 # Print result
115 print("Item Available" if found else "Item Not Available")
```

OUTPUT:



```
Item Available
```

EXPLANATION:

The original code used a loop to check each item in the list one by one and set a flag variable when the item was found. While this works, it makes the code longer. The improved version uses Python's `in` operator, which directly checks whether an item exists in the list. This makes the code shorter, easier to read, and more efficient while producing the same output: Item Available.